Keywords: evolutionary optimization • latent agents • program synthesis • scientific computing

SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization

Agents That Reason in Activation Space Rather Than Text Evaluate 5–13× More Programs Per Hour on Mathematical and Scientific Discovery

By Qile Jiang, George Em Karniadakis

arXiv:2611.08483 • cs.AI / cs.NE • Brown University

LLM-guided evolutionary search is accurate but slow because each candidate requires a full chain-of-thought forward pass. SwarmEvolve replaces text-decoding agents with latent agents that share KV caches and reason in activation space, evaluating 5–13× more programs per hour than state-of-the-art frameworks and reaching equivalent solution quality in 3–8 minutes versus 60 minutes for competing systems.

AT A GLANCE

Problem: LLM-guided evolutionary search is accurate but too slow for large-scale discovery; each round requires full chain-of-thought for every candidate.

Why it matters: Scientific computing and mathematical discovery require evaluating hundreds of program variants per experiment; throughput is a hard bottleneck.

Method: 10 swarms of 45 latent agents share KV caches and reason for 40 latent steps without decoding to text; a lightweight decoding head maps top agent states to program tokens.

Result: 5–13× more programs per hour; solution quality reached in 3–8 min vs. 60 min for text-based frameworks.

Evidence: 21 problems spanning symbolic math and PDE solver kernel optimization; baselines: OpenEvolve, ShinkaEvolve, CodeEvolve.

The Big Picture

Program evolution with LLMs (OpenEvolve, FunSearch, ShinkaEvolve) has produced surprising discoveries in combinatorics and numerical computing by using LLMs as intelligent mutation operators: given a population of programs and their fitness scores, the LLM proposes new variants that improve on the best candidates. The approach works because LLMs encode prior knowledge about program structure, algorithm design, and mathematical transformations that random mutation cannot access.

The fundamental bottleneck is throughput. Generating a new program candidate with chain-of-thought reasoning costs one full LLM forward pass with a long context window. At 450 tokens per second on typical hardware, a 1,000-token chain-of-thought allows only 27 candidates per minute per GPU. Scientific computing problems may need thousands of candidates before a significant improvement is found.

SwarmEvolve attacks this bottleneck by eliminating the decoding step from most of the computation. Agents in SwarmEvolve do not produce natural language — they produce latent representations and hand them to a compact decoding head. The decoding head is cheap, while the latent reasoning is parallelized across 450 agents per round.

Why Existing Approaches Fall Short

Text-based evolutionary LLM frameworks (OpenEvolve, CodeEvolve) generate each candidate as a full text completion. This is flexible but slow: each candidate occupies the full generation path of the LLM.

Parallel sampling with temperature can explore multiple candidates simultaneously, but each sample is independent and cannot condition on what other concurrent samples are producing, limiting diversity within a batch.

Speculative decoding speeds up text generation but does not reduce the number of distinct candidates that can be evaluated per unit time, since it optimizes throughput on a single sequence.

Smaller surrogate models are faster but lack the reasoning quality of large LLMs, leading to lower-quality proposals and slower convergence in solution quality.

Core Mechanism: Latent Agents with KV Sharing

Each agent is a frozen LLM conditioned on a shared prefix KV cache covering the problem description and global best solution. The agent then runs 40 autoregressive steps *without* applying the unembedding matrix: no token is produced, and no text is decoded. Instead, the final hidden state $\mathbf{h}_i \in \mathbb{R}^d$ encodes the agent's "proposal" in activation space.

Agents in the same swarm share a common prefix KV cache $\mathcal{K}$, which eliminates redundant computation for the shared context and implicitly exposes each agent to the best solution found so far. Agents have different random seeds (different initial noise vectors appended to the shared prefix), inducing diversity within the swarm.

A lightweight decoding head $D_\phi : \mathbb{R}^d \to \mathcal{V}^L$ maps the hidden states of the top-r agents (by cosine similarity to the fitness-weighted centroid of the swarm) to program

token sequences of length at most $L = 200$ tokens.

Technical Formulation

Let $\mathcal{S} = \{S_1, \ldots, S_K\}$ denote $K = 10$ swarms, each containing $N = 45$ agents. The shared prefix for swarm S_k is:

$$\mathcal{K}_k = \text{KVCache}\big([\text{problem}, \mathbf{x}_k^*]\big),$$

where $\mathbf{x}_k^*$ is the globally best program found in rounds $1, \ldots, t-1$ of swarm k.

Each agent $a_{k,i}$ produces a hidden state:

$$\mathbf{h}_{k,i} = f_\theta\big(\mathbf{z}_{k,i}, \mathcal{K}_k\big) \in \mathbb{R}^d,$$

where $\mathbf{z}_{k,i} \sim \mathcal{N}(0, \sigma^2 I)$ is the agent-specific noise vector and f_θ runs 40 latent steps.

The swarm centroid weighted by fitness is:

$$\bar{\mathbf{h}}_k = \frac{\sum_{i=1}^{N} w_i \mathbf{h}_{k,i}}{\sum_{i=1}^{N} w_i}, \quad w_i = \text{softmax}(F_{k,i}/T),$$

where $F_{k,i}$ is the previous-round fitness of agent $a_{k,i}$ and T is a temperature parameter. The top-$r = 5$ agents (by cosine similarity to $\bar{\mathbf{h}}_k$) are decoded by D_ϕ.

Learning or Inference Procedure

D_ϕ is a 2-layer MLP trained offline on (hidden-state, program) pairs extracted from text-based evolutionary runs. At inference time, D_ϕ is frozen; SwarmEvolve requires no online training.

The evolutionary loop per round:

1. Update all K prefix caches with current best solutions.
2. Run all $K \times N = 450$ agents in parallel (each 40 latent steps) on a single A100 node.
3. Select top-r agents per swarm using the fitness-weighted cosine similarity score.
4. Decode selected agents' hidden states to programs via D_ϕ.
5. Execute decoded programs in a sandboxed environment.
6. Update global best solutions for each swarm.

What the Guarantee Says

No formal convergence guarantee is provided. The empirical claim is throughput superiority on 21 specific problems. The best-known solutions to 11 of the 21 problems (symbolic math subset) have formal certificates; SwarmEvolve recovers certified-correct solutions for 9 of these 11 within the allocated time budget of 2 GPU-hours.

Experimental Findings

- **Throughput:** 5–13× more programs per hour than all three baselines on all 21 problems.
- **Time-to-solution:** Median 3–8 minutes to reach the quality of the best competing solution; baselines require 60 minutes on the same hardware.
- **Solution quality:** Highest average score across 21 problems; wins 17 of 21 head-to-head comparisons vs. the best single baseline per problem.
- **Diversity:** The 10-swarm configuration produces 1.9× more distinct program families per hour than a 450-agent single-swarm variant, confirming the value of swarm isolation for population diversity.

Ablations and Interpretation

Latent steps: 40 latent steps is near-optimal; 20 steps reduces quality by 4.3 average points; 80 steps shows no improvement while doubling per-round time.

Swarm count: 10 swarms outperform 5 swarms by 2.7 average points and 1 swarm by 7.1 points, confirming that swarm diversity is the main quality driver.

KV sharing: Removing the shared prefix KV cache (agents start from scratch each round) reduces throughput by 2.3× and quality by 3.2 points, since agents lose access to the current best solution.

Decoding head quality: Replacing D_ϕ with a linear probe reduces program validity (fraction of programs that compile and run) from 94% to 71%, confirming the need for a non-linear decoder.

Reference

Qile Jiang, George Em Karniadakis. **SwarmEvolve: Latent Agent Swarms for Evolutionary Discovery and Optimization.** arXiv:2611.08483. https://arxiv.org/abs/2611.08483